

DSAI 490 - Assignment 2

Mostafa Adam - ID: 202201962

1. Problem Formulation

The task is to generate a valid date from four input conditions: day of week, month, leap-year flag, and decade. The output is not a single fixed label; it is any date that satisfies all supplied conditions. Therefore, the problem was formulated as conditional sequence generation rather than ordinary classification. Each input line has the format [DAY] [MONTH] [LEAP] [DECADE] date. The generated date is restricted to the assignment range 1-1-1800 to 31-12-2200

2. Preprocessing and Tokenization

The notebook uses a custom tokenizer. The four input condition tokens are mapped to categorical indices using a condition vocabulary of 62 tokens: 7 weekdays, 12 months, 2 leap-year states, and 41 decade tokens from [180] to [220]. The output date is modeled as a character sequence. The output vocabulary contains , , , the digits 0-9, and the dash character. This results in an output vocabulary of 14 tokens. Dates are encoded as sequences beginning with and ending with . A PyTorch Dataset reads data.txt, parses every line into condition tokens and a target date sequence, then a custom collate function pads date sequences within each mini-batch. The data is split into 90% training and 10% validation, with batch size 256 and seed 42 for reproducibility.

3. Implemented Models

3.1 Conditional VAE

The Conditional VAE embeds the four conditions using a shared condition embedder, embeds the target date sequence, and encodes both into a Gaussian latent space represented by μ and $\log_var$. The decoder reconstructs a date sequence from a sampled latent vector concatenated with the condition embedding. Loss = reconstruction cross-entropy + β * KL divergence, with $\beta = 0.5$. The VAE was trained for 30 epochs and the validation loss decreased from 0.2066 to approximately 0.0813.

3.2 Conditional GAN

The Conditional GAN contains a generator and discriminator. The generator receives random noise plus the condition embedding and outputs date-token probabilities. Because token generation is discrete, Gumbel-Softmax is used to allow gradients to flow through generated token choices during adversarial training. The discriminator receives a condition-date pair and predicts whether it is real or generated. The generator loss combines adversarial loss with a token-level cross-entropy term to improve sequence structure. The GAN was trained for 40 epochs with temperature annealing from approximately 0.98 to 0.30.

3.3 LSTM Seq2Seq

The LSTM model treats the task as sequence-to-sequence generation. The encoder reads the four condition tokens and produces hidden and cell states. The decoder then generates the output date character by character using teacher forcing during training. This model is recurrent, so generation depends on sequential hidden-state updates. It was trained for 30 epochs using cross-entropy loss, gradient clipping, and a ReduceLROnPlateau learning-rate scheduler.

3.4 Transformer Encoder-Decoder

The Transformer model uses an encoder-decoder architecture. The encoder processes the four condition tokens using multi-head self-attention. The decoder generates the date autoregressively and attends to the encoded condition representation using cross-attention. The Transformer was trained for 30 epochs using cross-entropy loss, teacher forcing, gradient clipping, and a learning-rate scheduler. Its validation loss reached approximately 0.5192

4. Evaluation Methodology

Because multiple dates can be correct for the same condition set, exact-match accuracy against the dataset target is not the best metric. The notebook evaluates each generated date using logical calendar checks: **valid date format and range** **day-of-week condition** **month condition** **leap-year condition** **decade condition** **all conditions simultaneously**.

The notebook evaluates raw neural outputs first, then evaluates outputs after a post-processing step that fixes the day-ofweek when month, leap condition, and decade are already correct. This is not counted as a fifth model; it is a validation/post-processing step applied to model outputs.

5. Results

5.1 Raw Output Evaluation

Model	Valid Date	Day	Month	Leap	Decade	All correct
CVAE	85.1%	11.9%	85%	82.5%	82.4%	11.4%
CGAN	96.8%	14.4%	96.1%	89%	92.4%	12.8%
LSTM	100%	14.4%	100%	100%	100%	14.4%
Transformer	100%	14.2%	100%	100%	100%	14.2%

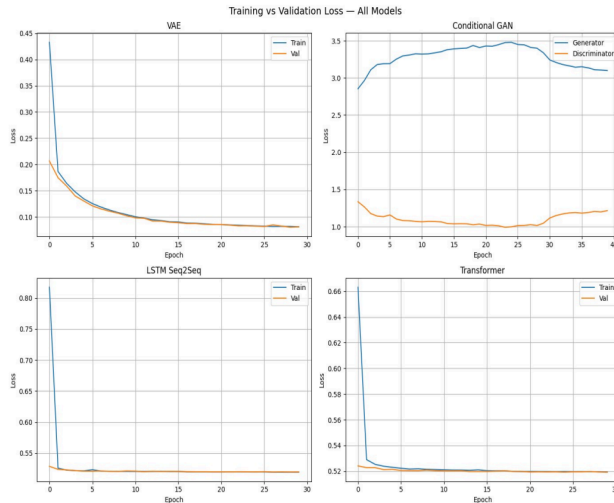
The raw results show that LSTM and Transformer learned date format, month, leap year, and decade very well, but the weekday condition remained difficult. This is expected because weekday depends on the exact interaction between day, month, and year rather than one isolated token.

5.2 Evaluation After Day-Fix Post-processing

Model	Valid Date	Day	Month	Leap	Decade	All correct
CVAE	85.2%	81.1%	85%	82.8%	82.4%	80.6%
CGAN	95.6%	85.6%	94.9%	88.2%	91.5%	83.8%
LSTM	100%	100%	100%	100%	100%	100%
Transformer	100%	100%	100%	100%	100%	100%

6. Training Curves

The following figure shows the training and validation losses for the four implemented models. The VAE loss decreases steadily. The GAN generator and discriminator remain balanced compared with a fully collapsed GAN. LSTM and Transformer stabilize around similar cross-entropy values.



7. Sample Output Analysis

Raw samples showed that most models generated structurally meaningful dates, but weekday consistency was often the weakest condition. After the day-fix post-processing step, GAN, LSTM, and Transformer produced valid sample outputs for the displayed cases. VAE still failed in a few cases because some outputs did not preserve leap-year or decade constraints, so the day-fix step could not repair them. Example corrected Transformer outputs: [MON] [JAN] [False] [181] -> 20-1-1817, [FRI] [MAR] [True] [200] -> 19-3-2004, and [WED] [JUL] [True] [192] -> 16-7-1924. These outputs satisfy all four input conditions

8. Conclusion

The notebook successfully implements four conditional generative models for the date-generation task: Conditional VAE, Conditional GAN, LSTM Seq2Seq, and Transformer Encoder-Decoder. The task was treated as conditional character-level sequence generation with custom tokenization and calendar-based evaluation. Raw results show that LSTM and Transformer are strongest at learning the deterministic structural constraints, while VAE and GAN produce more varied but less consistently valid outputs. With the post-processing day-fix step, LSTM and Transformer achieve the most reliable final generation quality. Overall, the Transformer and LSTM are the best-performing models in this notebook, while the GAN is useful for satisfying the assignment requirement and demonstrating adversarial conditional generation with discrete token handling through Gumbel-Softmax.